

CrediSense AI

Guide de déploiement sur Azure : ce qui a été fait, comment le refaire, comment le tester

Date : 5 octobre 2026 · **Dépôt** : github.com/meriem277/CrediSense-AI (branche `deploy/azure`)

Application : <https://135-225-76-98.nip.io> · **VM** : 135.225.76.98 (Azure, Suède centre)

Souscription : Azure for Students · **État au moment de la rédaction** : les 5 conteneurs tournent sur la VM ; les tests fonctionnels détaillés au §5 restent à dérouler.

Sommaire

1. Architecture déployée
2. Ce qui a été fait, dans l'ordre
3. Procédure complète pour refaire le déploiement
4. Problèmes rencontrés, causes et corrections
5. Comment tester l'application
6. Ce qui reste à faire (sécurité et qualité)
7. Commandes utiles au quotidien
8. Mettre à jour l'application après des modifications
9. Alternatives étudiées et décisions prises

1. Architecture déployée

Élément	Valeur
Région Azure	<code>swedencentral</code> (la politique de la souscription n'autorise que <code>switzerlandnorth</code> , <code>spaincentral</code> , <code>swedencentral</code> , <code>norwayeast</code> , <code>austriaeast</code>)
VM	<code>Standard_B2as_v2</code> : 2 vCPU, 8 Go de RAM, Ubuntu 22.04, disque 64 Go
Groupe de ressources	<code>credisense-rg</code> (créé par Terraform)
Réseau	IP publique statique, VNet, sous-réseau, NSG (ports 22, 80, 443 ouverts)
Conteneurs (Docker Compose)	<code>postgres</code> (base de données), <code>ai</code> (FastAPI + PaddleOCR 3.x, port 8002), <code>backend</code> (Spring Boot, port 8081), <code>frontend</code> (Angular SSR, port 4000), <code>caddy</code> (HTTPS, ports 80 et 443)
Routage Caddy	<code>/api/*</code> vers le backend ; tout le reste vers le frontend. Certificat Let's Encrypt automatique pour <code>135-225-76-98.nip.io</code>
Images Docker	<code>ghcr.io/meriem277/credisense-{ai,backend,frontend}:latest</code> , construites directement sur la VM
Fichiers de configuration sur la VM	<code>/opt/credisense/.env</code> et <code>ai.env</code> (écrits par Terraform/cloud-init)

Les ressources Azure sont décrites dans `infra/terraform/`. Le service OCR externe *Document Intelligence* (groupe `pfemeriem-rg`) est une ressource à part, que ce projet n'utilise pas.

2. Ce qui a été fait, dans l'ordre

2.1 Audit du projet

Un audit du dépôt a relevé les points principaux suivants (détail au §6) : clés et mots de passe présents dans l'historique git, données personnelles de clients versionnées (`Backend/uploads/`), routes backend publiques (`permitAll`) et lecture de fichiers sans authentification, endpoint `/health` de l'IA cassé, fichier `requirements.txt` incohérent avec le code.

2.2 Création de l'infrastructure avec Terraform (Cloud Shell)

- Premier essai en `uksouth` : refusé par la politique Azure (`RequestDisallowedByAzure`). Cette région n'est pas autorisée sur la souscription.
- Liste des régions autorisées lue avec `az policy assignment list`. Aucune restriction sur les tailles de VM ni sur les types de ressources.
- Quotas de vCPU lus avec `az vm list-usage` : famille D v5 à 0, familles BS (4 vCPU), Basv2 et Bsv2 (10 vCPU) disponibles.
- Échecs successifs de capacité (`SkuNotAvailable`) pour `Standard_B2ms` et `B4ms` en `norwayeast`. Les B-series n'existent pas à `spaincentral`, `switzerlandnorth` ni `austriaeast` (vérifié avec `Get-AzComputeResourceSku`).
- Combinaison retenue qui a fonctionné : `swedencentral` + `Standard_B2as_v2`.
- Plusieurs erreurs liées à l'état Terraform (ressources créées par Azure mais absentes du state : « already exists », « Provider produced inconsistent result »). Corrigées en important les ressources avec `terraform import`.

2.3 Mise à jour du code Terraform

- `variables.tf` : valeurs par défaut `swedencentral` et `Standard_B2as_v2`, descriptions mises à jour.
- `terraform.tfvars.example` : notes sur les régions et les familles de VM.
- `recover-state.sh` (nouveau) : importe dans le state les ressources qui existent déjà dans Azure.

2.4 Connexion de GitHub à la VM

- `gh auth login` dans Cloud Shell.
- Secret GitHub `SSH_PRIVATE_KEY` (clé générée par Terraform) et variable `AZURE_VM_IP` créés avec `gh secret set` et `gh variable set`.

2.5 Mise en conformité de l'image Docker du service IA

Le build de l'image IA échouait sur GitHub Actions. Les causes identifiées sont au §4. Corrections apportées dans `Python/requirements.txt` et `Python/Dockerfile` :

- `paddleocr==3.4.0` et `paddlepaddle==3.3.1` (le code `ocr_service.py` utilise l'API 3.x ; les anciennes versions épinglées étaient 2.8.1 et 2.6.2).
- Préchargement des modèles PaddleOCR avec l'API 3.x (modèles `PP-OCRv5` : détection, arabe, latin).

- `pyclipper==1.3.0.post6`, `scipy==1.13.1`, `scikit-learn==1.5.2` épinglés ; `LD_PRELOAD` sur le `zlib` système ; base `python:3.11-slim-bookworm` ; étape de contrôle des dépendances natives au build.

Ces modifications ont été publiées sur la branche `deploy/azure` (commit `09fd0c1e`).

2.6 Contournement des blocages de GitHub Actions

Les jobs du workflow restaient longtemps en attente d'un runner (« Waiting for a runner to pick up this job »), parfois plus de 15 minutes. Ce n'est pas une erreur de code. Pour avancer sans dépendre de cette file, les images ont été **construites directement sur la VM** (§3, étapes 7 à 9).

2.7 Démarrage de l'application

- Fichiers `docker-compose.prod.yml` et `Caddyfile` copiés sur la VM avec `scp`.
- Images backend, frontend et IA construites sur la VM sous les noms attendus par le compose.
- Image publique `caddy:2-alpine` téléchargée à part (l'option `--pull never` empêche de la télécharger automatiquement).
- `docker compose up -d --pull never` : les 5 conteneurs sont démarrés.

3. Procédure complète pour refaire le déploiement

Dans l'ordre. Chaque bloc de commandes est à lancer dans le terminal indiqué.

Étape 1 : choisir la région et la taille de VM (Cloud Shell)

```
az policy assignment list --query "[].parameters.listOfAllowedLocations.value" -o json
az vm list-usage --location swedencentral --query "[?limit!='0'].{famille:name.localizedValue,
limite:limit}" -o table
```

Étape 2 : cloner le dépôt et préparer Terraform (Cloud Shell)

```
git clone https://github.com/meriem277/CrediSense-AI.git
cd CrediSense-AI && git checkout deploy/azure
cd infra/terraform
cp terraform.tfvars.example terraform.tfvars # puis éditer : image_prefix, ai_env,
google_client_id, smtp_*
# ajouter aussi :
# location = "swedencentral"
# vm_size = "Standard_B2as_v2"
```

Ne jamais committer `terraform.tfvars` : il contient des clés d'API et des mots de passe (il est déjà dans le `.gitignore`).

Étape 3 : créer l'infrastructure (Cloud Shell)

```
terraform init
terraform apply -parallelism=1 -no-color 2>&1 | tee ~/apply.log
```

Si une erreur dit qu'une ressource « already exists » ou que le provider a produit un « résultat incohérent », lancer `bash recover-state.sh` puis relancer `terraform apply`. Si Azure répond `SkuNotAvailable`, essayer une

autre taille ou une autre région. Quand on change de région, détruire d'abord (`terraform destroy`) pour éviter des conflits de réseau.

Étape 4 : récupérer les sorties (Cloud Shell)

```
terraform output app_url
terraform output public_ip
terraform output -raw ssh_private_key > ~/vm_key && chmod 600 ~/vm_key
```

Étape 5 : donner à GitHub l'accès à la VM (Cloud Shell)

```
gh auth login
terraform output -raw ssh_private_key | gh secret set SSH_PRIVATE_KEY -R meriem277/CrediSense-AI
gh variable set AZURE_VM_IP -R meriem277/CrediSense-AI --body "$(terraform output -raw public_ip)"
```

Étape 6 : copier les fichiers de déploiement et se connecter (Cloud Shell)

```
scp -i ~/vm_key ../../deploy/docker-compose.prod.yml ../../deploy/Caddyfile
credisense@<IP>:/opt/credisense/
ssh -i ~/vm_key credisense@<IP>
```

Étape 7 : vérifier que la VM est prête (sur la VM)

```
cloud-init status --wait      # doit afficher : status: done
docker --version
docker ps                     # un conteneur postgres est déjà lancé
```

Étape 8 : récupérer le code et construire les images (sur la VM)

```
git clone --branch deploy/azure https://github.com/meriem277/CrediSense-AI.git ~/src
cd ~/src
docker build -t ghcr.io/meriem277/credisense-backend:latest Backend/backend
docker build -t ghcr.io/meriem277/credisense-frontend:latest frontend
nohup docker build --progress=plain -t ghcr.io/meriem277/credisense-ai:latest Python > ~/ai-build.log 2>&1 &
```

Le build IA prend environ 10 à 20 minutes. Suivre l'avancement avec `tail -n 30 ~/ai-build.log`.

Étape 9 : démarrer l'application (sur la VM)

```
cd /opt/credisense
grep -E '^IMAGE_' .env      # IMAGE_PREFIX=ghcr.io/meriem277/credisense et IMAGE_TAG=latest
docker pull caddy:2-alpine
docker compose -f docker-compose.prod.yml up -d --pull never
docker compose -f docker-compose.prod.yml ps
```

Étape 10 : créer l'administrateur

```
curl -i -X POST https://<domaine>/api/auth/init-admin \
-H "Content-Type: application/json" \
-d '{"nom":"Admin","email":"votre.email@exemple.com","password":"UnMotDePasseSolide!"}'
```

Fonctionne une seule fois : dès qu'un administrateur existe, l'endpoint répond 403.

4. Problèmes rencontrés, causes et corrections

Symptôme	Cause	Correction
<code>RequestDisallowedByAzure</code>	Région non autorisée par la politique de la souscription étudiante	Utiliser une région de la liste : <code>swedencentral</code> , <code>norwayeast</code> , <code>switzerlandnorth</code> , <code>spaincentral</code> , <code>austriaeast</code>
<code>SkuNotAvailable</code>	Pas de capacité Azure pour cette taille, à ce moment, dans cette région	Essayer une autre taille d'une famille où il y a du quota, ou une autre région ; les B-series n'existent pas dans toutes les régions
<code>InUseSubnetCannotBeDeleted</code>	Changement de région sans détruire d'abord les ressources précédentes	<code>terraform destroy</code> avant de changer de région
<code>already exists ... needs to be imported</code>	Ressource créée dans Azure mais absente du state Terraform (souvent après plusieurs commandes Terraform en parallèle)	<code>terraform import</code> , ou <code>recover-state.sh</code> ; ne jamais lancer deux commandes Terraform en même temps
<code>Provider produced inconsistent result after apply</code>	Azure expose la ressource avec un léger retard, le provider la croit absente	Importer la ressource, puis relancer <code>terraform apply</code>
Build IA : <code>zlib.error: inconsistent stream state</code>	Paquets récents compilés avec une version de Cython qui compresse ses constantes (<code>pyclipper 1.4.0</code> , <code>scipy 1.17</code>), en conflit une fois <code>paddle</code> chargé. Explication probable, non prouvée	Versions plus anciennes épinglées (<code>pyclipper 1.3.0</code> , <code>scipy 1.13.1</code> , <code>scikit-learn 1.5.2</code>) et <code>LD_PRELOAD</code> sur le <code>zlib</code> système
Build IA incohérent avec le code	<code>requirements.txt</code> épinglait <code>PaddleOCR 2.8.1</code> alors que <code>ocr_service.py</code> utilise l'API 3.x	Passage à <code>paddleocr==3.4.0</code> et <code>paddlepaddle==3.3.1</code> et préchargement des modèles 3.x
Jobs GitHub Actions « Waiting for a runner »	Capacité des runners hébergés par GitHub	Build sur la VM elle-même ; relancer les jobs avec <code>gh run rerun <id> --failed</code> quand le run est terminé

Cannot GET / sur la page d'accueil	Le routeur Angular n'a aucune route pour / : la redirection vers login et la route ** sont commentées dans frontend/src/app/app.routes.ts . Avec le rendu serveur (SSR), Express répond alors 404. Le déploiement lui-même est correct	Ouvrir directement /login ou /client/login . Pour corriger : réactiver { path: '', redirectTo: 'login', pathMatch: 'full' } dans app.routes.ts , puis reconstruire l'image frontend
No such image: caddy:2-alpine	--pull never empêche aussi de télécharger les images publiques	docker pull caddy:2-alpine avant docker compose up

5. Comment tester l'application

5.1 Santé des services

```
curl -I https://135-225-76-98.nip.io/login      # frontend : attendu 200
curl -i https://135-225-76-98.nip.io/api/auth/login  # backend : 405 ou 400 = il répond ; 502 = pas prêt
```

La racine / répond Cannot GET / tant que la route d'accueil n'est pas définie dans l'application Angular (voir §4) : tester plutôt /login et /client/login . Le premier certificat HTTPS peut demander une minute. Le service IA a besoin de 2 à 3 minutes après le démarrage pour charger ses modèles.

5.2 Scénarios fonctionnels

1. Créer l'administrateur (§3, étape 10), puis se connecter dans le navigateur.
2. Créer un agent depuis le compte administrateur.
3. Créer un client et un dossier de crédit.
4. Téléverser un document : PDF texte, puis PDF scanné (CIN, attestation), pour tester l'extraction de texte et l'OCR.
5. Lancer l'analyse du dossier : classification des documents, extraction des informations, score.
6. Tester le chatbot sur un dossier indexé.
7. Tester « mot de passe oublié » (nécessite les identifiants SMTP renseignés dans terraform.tfvars).
8. Tester la connexion Google, après avoir ajouté l'URL de l'application aux origines JavaScript autorisées du client OAuth.

5.3 Diagnostic en cas de problème (sur la VM)

```
cd /opt/credisense
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs backend --tail 50
docker compose -f docker-compose.prod.yml logs ai --tail 50
docker compose -f docker-compose.prod.yml logs caddy --tail 20
free -h      # mémoire : 8 Go au total pour toute la pile
df -h /      # espace disque
```

6. Ce qui reste à faire

Priorité	Point	Détail
Urgent	Régénérer les clés et mots de passe	Clés Groq et Mistral, mot de passe d'application Gmail, mot de passe de la base, secret JWT : ils sont apparus dans l'historique git et dans des conversations. Les révoquer puis mettre de nouvelles valeurs dans <code>terraform.tfvars</code>
Urgent	Données personnelles dans git	Backend/uploads/client-uploads/ (environ 389 fichiers : CIN, fiches de paie, relevés) est suivi par git et publié sur GitHub. Retirer avec <code>git rm -r --cached</code> , ajouter au <code>.gitignore</code> , envisager une réécriture de l'historique
Urgent	Routes backend sans authentification	<code>SecurityConfig.java</code> met <code>/api/clients/**</code> , <code>/api/fichiers/**</code> (y compris la suppression) et <code>/api/chatbot/**</code> en accès public. <code>/api/fichiers/view/**</code> permet de lire n'importe quel fichier du serveur
Important	Valeurs par défaut sensibles	<code>application.properties</code> donne une valeur par défaut au secret JWT et à d'autres secrets ; les retirer pour que l'application refuse de démarrer sans configuration
Important	Endpoint <code>/health</code> du service IA	Python/main.py ligne 88 lit <code>_doctr_model</code> , qui n'existe plus : le conteneur IA sera marqué <code>unhealthy</code> alors qu'il fonctionne
Important	Workflow GitHub Actions	Non validé de bout en bout : les derniers runs ont été annulés ou ont échoué avant le déploiement. Quand le workflow déploie, il remplace <code>IMAGE_TAG</code> par l'identifiant du commit : ne pas le lancer pendant un test local sans avoir vérifié que les images de ce commit existent sur GHCR
Important	SSH ouvert à tout Internet	Port 22 ouvert à <code>*</code> (authentification par clé uniquement). Nécessaire pour les déploiements GitHub Actions, mais à restreindre si possible
Qualité	Tests et dépendances	Pas de tests automatisés ; <code>pdfbox</code> et <code>poi-ooxml</code> déclarés en double dans <code>pom.xml</code> ; <code>requirements.txt</code> à la racine est un export <code>pip freeze</code> d'un autre environnement ; <code>@CrossOrigin</code> redondants dans les contrôleurs

7. Commandes utiles au quotidien

Besoin	Commande
Arrêter la VM (arrête la facturation de la machine)	<code>az vm deallocate -g credisense-rg -n credisense-vm</code>
Redémarrer la VM	<code>az vm start -g credisense-rg -n credisense-vm</code> (l'IP publique est statique)
Redémarrer un service	<code>docker compose -f docker-compose.prod.yml restart backend</code>
Reconstruire une image après une modification	<code>docker build -t ghcr.io/meriem277/credisense-backend:latest Backend/backend</code> puis <code>docker compose -f docker-compose.prod.yml up -d --pull never</code>
Tout supprimer côté Azure	<code>terraform destroy</code> depuis <code>infra/terraform/</code> (supprime la VM et le réseau ; irréversible)

Relancer seulement les jobs GitHub échoués

```
gh run rerun <id> --failed -R meriem277/CrediSense-AI
```

8. Mettre à jour l'application après des modifications

On ne refait **pas** tout le processus. Le déploiement initial (§3) est à faire une seule fois.

8.1 Ce qui reste en place

- La VM, le réseau et l'IP publique créés par Terraform : `terraform apply` n'est relancé que si l'infrastructure elle-même change (taille de VM, ports ouverts, etc.).
- Le secret `SSH_PRIVATE_KEY` et la variable `AZURE_VM_IP` de GitHub.
- Les données : la base PostgreSQL et les fichiers téléversés sont dans des volumes Docker (`credisense_uploads` et le volume de Postgres) et survivent aux mises à jour des conteneurs.

8.2 Chemin 1 : mise à jour manuelle sur la VM (méthode validée)

```
# Sur le PC : modifier, committer, pousser
git add . && git commit -m "description du changement" && git push origin deploy/azure

# Sur la VM : ssh -i ~/vm_key credisense@135.225.76.98
cd ~/src && git pull
docker build -t ghcr.io/meriem277/credisense-backend:latest Backend/backend # uniquement le service modifié
cd /opt/credisense
docker compose -f docker-compose.prod.yml up -d --pull never backend
```

Service modifié	Dossier à construire	Nom de l'image	Durée approximative
backend	Backend/backend	credisense-backend:latest	quelques minutes
frontend	frontend	credisense-frontend:latest	environ 1 minute
ai	Python	credisense-ai:latest	10 à 20 minutes la première fois ; plus court si seules les sources changent (couches de dépendances en cache)

Avec `up -d --pull never <service>`, seul ce service redémarre ; les autres continuent de tourner. Pour l'IA, laisser 2 à 3 minutes pour le rechargement des modèles.

8.3 Chemin 2 : mise à jour automatique avec GitHub Actions

Un `git push` sur `deploy/azure` déclenche le workflow `deploy.yml` : construction des trois images, envoi vers GHCR, puis déploiement sur la VM. Ce chemin n'a pas été validé de bout en bout (attentes de runner côté GitHub).

Piège si on mélange les deux chemins. Quand le workflow déploie, il remplace `IMAGE_TAG` dans `/opt/credisense/.env` par l'identifiant du commit. Dès lors, `docker compose` utilise les images de GHCR et ignore celles construites à la main sous le tag `latest`. Pour revenir à la méthode manuelle :

```
sed -i 's|^IMAGE_TAG=.*|IMAGE_TAG=latest|' /opt/credisense/.env
```

Choisir un chemin et s'y tenir, ou remettre `IMAGE_TAG=latest` avant chaque build manuel.

8.4 Changer la configuration (clés d'API, SMTP, etc.)

- Les fichiers `/opt/credisense/.env` et `ai.env` sont écrits par Terraform au premier démarrage de la VM.
- Modifier `terraform.tfvars` puis relancer `terraform apply` peut **recréer la VM**, et donc faire perdre les données de la base : à ne pas faire sans avoir réfléchi et sauvegardé.
- Pour un simple changement de variable, il est plus sûr d'éditer directement `/opt/credisense/.env` ou `ai.env` sur la VM, puis d'exécuter `docker compose -f docker-compose.prod.yml up -d`.

8.5 Avant de pousser un changement

- Ne jamais committer de clés, de mots de passe ni de fichiers `.env`.
- Tester le changement en local si possible avant de reconstruire une image sur la VM.
- Après une mise à jour, vérifier : `docker compose -f docker-compose.prod.yml ps`, puis les journaux du service modifié.

9. Alternatives étudiées et décisions prises

Idée	Conclusion
Utiliser <code>uksouth</code> ou <code>Standard_D2s_v7</code> (config d'un autre projet)	Impossible : <code>uksouth</code> est interdite par la politique de la souscription, et la famille D v5 a un quota à 0. La VM retenue vient d'une famille où le quota existe (Basv2)
VM de 4 Go de RAM (B2s)	Déconseillée : Postgres, Spring Boot, le frontend et l'IA (PyTorch + PaddleOCR) demandent environ 8 Go. Possible seulement avec du swap et un service IA allégé (par exemple en remplaçant PaddleOCR par Azure Document Intelligence, déjà créé)
Conteneurs gérés (Container Instances, Container Apps, App Service) à la place d'une VM	Écartés : coût plus élevé pour des conteneurs toujours allumés, limites de mémoire, quotas souvent à 0 sur les comptes étudiants, et réécriture complète de Terraform et du déploiement
Séparer les trois builds en trois workflows	Écarté : chaque job demande déjà son propre runner dans la file de GitHub, et le déploiement a besoin des trois images au même identifiant de commit. Pour relancer seulement un job échoué : <code>gh run rerun <id> --failed</code>
Remplacer <code>ubuntu-latest</code> par une version fixe (<code>ubuntu-24.04</code>)	Essayé dans <code>deploy.yml</code> pour limiter l'instabilité du label <code>ubuntu-latest</code> . Le fichier utilise actuellement <code>ubuntu-latest</code> ; l'effet sur les attentes n'a pas pu être démontré

Runner GitHub auto-hébergé	Possible (sur la VM de production ou sur une seconde VM), non mis en place : sur la VM de production, les builds ralentiraient l'application ; sur une seconde VM, coût supplémentaire. À éviter sur un dépôt public pour des raisons de sécurité
Construire les images sur le PC puis les envoyer vers GHCR	Non retenu : Docker n'était pas disponible sur le PC, et l'envoi de plusieurs Go est lent. La construction sur la VM évite le registre et l'attente de GitHub

Ce document ne contient aucun secret. Les clés, mots de passe et la clé SSH privée se trouvent dans `terraform.tfvars` (non versionné), dans le state Terraform et dans les secrets GitHub.